

Amnésia: Análise de Desempenho da Memória Virtual e da Cache

Ana Fernanda Souza Cancado*, Arthur de Sá Braz de Matos*, Gabriel Praes B Nunes*,
Guilherme Otávio de Oliveira*, Kaio Henrique Santos*

Abstract—Este trabalho analisa o impacto de diferentes parâmetros e políticas no desempenho de sistemas de memória virtual e cache. Foram avaliadas as políticas de substituição FIFO e LRU, o tamanho de página, o tamanho da TLB, o número de páginas, o tamanho da cache, o tamanho da linha da cache e a associatividade. Utilizando diferentes traces de acesso à memória para explorar diferentes cenários, como acessos sequenciais, repetidos e aleatórios. Diferentes parâmetros influenciam significativamente o desempenho até um ponto de saturação.

Index Terms—Memória Virtual, Cache, Políticas de Substituição, FIFO, LRU, TLB, Tamanho de Página, Tamanho de Linha, Associatividade, Desempenho de Memória.

I. INTRODUÇÃO

O gerenciamento eficiente da memória é crucial para o desempenho de sistemas computacionais modernos. A memória virtual e a memória cache são mecanismos fundamentais que visam otimizar o acesso aos dados, conciliando velocidade, capacidade e custo. Este trabalho investiga o impacto de diversas configurações e políticas de gerenciamento nesses dois níveis de hierarquia de memória. Serão analisados experimentalmente os efeitos do tamanho de página, das políticas de substituição (FIFO e LRU), do tamanho da Translation Lookaside Buffer (TLB) e do número de páginas na memória virtual. Em seguida, será avaliado o desempenho da memória cache em relação às políticas de substituição, ao tamanho da cache, ao tamanho da linha e ao nível de associatividade. O objetivo é identificar as configurações que proporcionam melhor desempenho sob diferentes padrões de acesso à memória.

O conceito de localidade é fundamental para entender o comportamento de acesso à memória e otimizar o desempenho dos caches. Existem dois principais tipos de localidade:

- **Localidade Temporal:** refere-se à tendência de um programa acessar novamente os mesmos dados em um curto intervalo de tempo. Ou seja, se uma informação foi acessada recentemente, é provável que ela seja acessada novamente em breve. Tamanhos maiores de memória podem favorecer este tipo de localidade.
- **Localidade Espacial:** está relacionada ao acesso sequencial de dados. Se uma posição de memória é acessada, é provável que posições próximas também sejam acessadas em breve. Por isso, mecanismos como pré-busca de blocos inteiros de memória são empregados para antecipar acessos e reduzir latências. Tamanhos de linha maiores podem favorecer este tipo de localidade.

II. TRABALHOS CORRELATOS

A análise do desempenho de memória virtual e cache é um tópico bem estabelecido na literatura de sistemas de computação. Diversos trabalhos exploram o impacto de diferentes políticas e parâmetros nas hierarquias de memória.

Wierzbicki *et al.* [1] investigaram políticas de substituição de cache, focando especificamente no contexto do tráfego peer-to-peer. Embora o foco da aplicação seja diferente do nosso trabalho, a análise comparativa de diferentes políticas de substituição, como FIFO e LRU, é relevante para a nossa avaliação do desempenho da cache. As conclusões sobre a eficácia de diferentes políticas sob cargas de trabalho específicas podem fornecer dados comparáveis aos nossos resultados com diferentes traces de acesso à memória.

Uma revisão sobre a otimização da memória virtual em ambientes de computação em nuvem foi realizada por [2]. Embora o contexto seja a virtualização em nuvem, os princípios de gerenciamento de memória virtual, incluindo o tamanho da página e as políticas de substituição, permanecem fundamentais. A discussão sobre os desafios e as técnicas para otimizar a memória virtual em ambientes virtualizados pode complementar nossa análise experimental sobre o impacto desses parâmetros no desempenho geral da memória virtual.

Uma abordagem para alcançar zero cache miss, baseada na análise do grafo de fluxo de controle, foi proposta por [3]. Este trabalho se concentra em uma otimização de cache mais específica e proativa, contrastando com nossa análise do impacto de parâmetros como tamanho da cache, tamanho da linha e associatividade sob diferentes padrões de acesso. No entanto, a motivação de melhorar o desempenho da memória cache e reduzir os misses é um objetivo comum.

Enquanto esses trabalhos correlatos abordam aspectos específicos do gerenciamento de memória virtual e cache, nosso trabalho busca fornecer uma análise experimental abrangente do impacto de vários parâmetros inter-relacionados em ambos os níveis da hierarquia de memória, utilizando diferentes padrões de acesso para simular diversos cenários de carga de trabalho. A análise comparativa das políticas de substituição (FIFO e LRU), tamanhos de página e TLB na memória virtual, juntamente com o tamanho da cache, tamanho da linha e associatividade na memória cache, sob diferentes traces de acesso (aleatório, temporal e espacial), contribui para uma compreensão mais abrangente do desempenho da hierarquia de memória.

III. METODOLOGIA

Para avaliar o desempenho de diferentes configurações de memória, utilizamos o simulador Amnesia, uma ferramenta desenvolvida para análise de hierarquias de memória, incluindo cache e memória virtual. O simulador foi configurado com os seguintes parâmetros:

- Tamanho da palavra (bytes): 4
- Política de escrita: WriteBack

Cenário	Parâmetros
Cenário 1.1	Tamanho do bloco (palavras): 1
	Tamanho da memória principal (palavras): 8
	Tamanho da página (palavras): 2 / 4 / 8
	Tamanho da memória secundária (palavras): 16
	Algoritmo de substituição: FIFO / LRU
Cenário 1.2	Tipo da TLB: sem TLB
	Tamanho do bloco (palavras): 1
	Tamanho da memória principal (palavras): 32
	Tamanho da página (palavras): 4
	Tamanho da memória secundária (palavras): 16
Cenário 1.3	Algoritmo de substituição: FIFO / LRU
	Tipo da TLB: unified
	Tamanho do bloco (palavras): 2
	Tamanho da memória principal (palavras): 128
	Tamanho da página (palavras): 2 / 4 / 8 / 16 / 32 / 64
Cenário 2.1	Tamanho da memória secundária (palavras): 64
	Algoritmo de substituição: FIFO
	Tipo da TLB: sem TLB
	Tamanho do bloco (palavras): 2
	Tamanho da memória principal (palavras): 32
Cenário 2.2	Tamanho da linha (palavras): 1
	Tamanho da memória cache (palavras): 4 / 8 / 16 / 32 / 64
	Algoritmo de substituição: FIFO / LRU
	Nível de associatividade: 2
	Tamanho do bloco (palavras): 1
Cenário 2.3	Tamanho da memória principal (palavras): 256
	Tamanho da linha (palavras): 1 / 2 / 4 / 8 / 16 / 32
	Tamanho da memória cache (palavras): 64
	Algoritmo de substituição: FIFO
	Nível de associatividade: 2
Cenário 2.3	Tamanho do bloco (palavras): 1
	Tamanho da memória principal (palavras): 8
	Tamanho da linha (palavras): 1
	Tamanho da memória cache (palavras): 4
	Algoritmo de substituição: LRU
Cenário 2.3	Nível de associatividade: 1 / 4 / 16 / 64

TABLE I
PARÂMETROS DOS COMPONENTES DE MEMÓRIA

A. Padrões de Acesso Analisados

Para analisar os resultados, foram empregados três perfis distintos de traces de acesso à memória. O primeiro padrão consistiu em acessos completamente aleatórios, sem qualquer localidade aparente, simulando cenários onde os endereços de memória são referenciados de forma imprevisível. O segundo perfil foi construído para apresentar forte localidade temporal, com repetição de conjuntos específicos de endereços. Por fim, o terceiro trace foi projetado para explorar localidade espacial, com sequências de acessos a endereços contíguos na memória, simulando o processamento de arrays e estruturas de dados sequenciais.

B. Processo de Simulação

O fluxo experimental seguiu uma metodologia em quatro etapas principais. Inicialmente, realizou-se a configuração dos

parâmetros arquiteturais, como tamanhos de cache, políticas de substituição e características da memória virtual. Na segunda etapa, o sistema carregava o trace de memória selecionado, convertendo os endereços lógicos em requisições simuladas. A fase de execução operava ciclo-a-ciclo, simulando cada acesso à memória e atualizando os estados internos da hierarquia de memória. Finalmente, na última etapa, o simulador coletava métricas de desempenho, como taxas de page faults, miss ratio na cache, e tempo total de execução, armazenando esses dados para posterior análise comparativa.

IV. ANÁLISE DE RESULTADOS

A. Memória Virtual

A memória virtual permite que processos utilizem um espaço de endereçamento maior do que a memória física disponível, combinando a memória principal com o armazenamento em disco. Isso é feito através da paginação, que organiza a memória em blocos menores chamados páginas. Quando a memória física fica cheia, páginas menos utilizadas são movidas para o disco para abrir espaço.

A eficiência da memória virtual depende de alguns fatores importantes. A política de substituição de páginas define quais páginas serão retiradas da memória, buscando minimizar o impacto no desempenho. O tamanho das páginas também influencia, tamanhos menores evitam desperdício, mas podem gerar mais trocas. Além disso, a TLB (Translation Lookaside Buffer) é usada para agilizar a tradução dos endereços virtuais, evitando consultas demoradas na tabela de páginas.

1) Cenário 1.1: Políticas de Substituição e Tamanho de Página: Neste cenário, avaliou-se o impacto das políticas de substituição First-In, First-Out (FIFO) e Least Recently Used (LRU), bem como do tamanho de página (2, 4 e 8 palavras), no número de page faults e no tempo total de execução do sistema. A configuração base considerou uma memória principal com capacidade para 8 palavras (32 bits) e uma memória secundária (disco) com 16 palavras (64 bits), criando um ambiente controlado para análise dos parâmetros de memória virtual.

O trace utilizado era composto por uma sequência de acessos aleatórios, sem padrão de repetição ou proximidade entre os endereços acessados. Devido a essa aleatoriedade, o comportamento do sistema ficou altamente sensível ao tamanho da página: como o acesso a endereços consecutivos era raro, páginas maiores tendiam a desperdiçar espaço com dados não utilizados, aumentando o número de page faults e o tempo de execução.

Os resultados obtidos, apresentados nas Figuras 1 e 2, indicam que um tamanho de página menor (2 palavras) foi o mais eficiente para o padrão de acesso observado. Com páginas pequenas, houve menor desperdício de memória e, conseqüentemente, redução nos page faults e no tempo necessário para completar a execução do trace. Além disso, a política LRU demonstrou desempenho superior ao FIFO, especialmente quando utilizada em conjunto com páginas

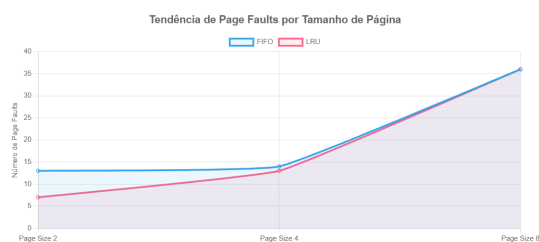


Fig. 1. Número de Page Faults por Tamanho de Página.

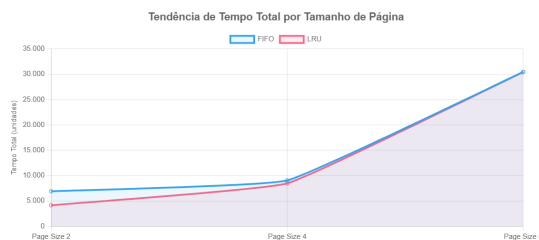


Fig. 2. Tempo de Page Faults por Tamanho de Página.

menores. Isso ocorre porque o LRU consegue preservar na memória as páginas acessadas mais recentemente, o que, mesmo em acessos aleatórios, oferece uma ligeira vantagem em comparação com o FIFO, que remove páginas apenas com base na ordem de entrada, sem considerar a frequência de uso.

Outro ponto observado foi o impacto do custo de cada page fault: à medida que o tamanho da página aumentava, o tempo necessário para lidar com uma falta também crescia, pois uma quantidade maior de dados precisava ser transferida entre a memória secundária e a principal. Assim, mesmo que o número absoluto de page faults pudesse, em alguns casos, reduzir levemente, o aumento do custo individual de cada falta acabou prejudicando o desempenho geral.

Esses resultados reforçam a importância de considerar não apenas o número de page faults, mas também o custo associado a cada uma, especialmente em sistemas onde o padrão de acesso não favorece a localidade e o tamanho da página se torna um fator crítico de desempenho.

2) Cenário 1.2: Tamanho da TLB: Este cenário investigou a relação entre o tamanho da TLB, a taxa de acerto (*hit rate*) e o tempo total de execução, comparando as políticas FIFO e LRU. O *trace* utilizado foi projetado para explorar a localidade temporal, com acessos repetidos a endereços em curtos intervalos, forçando substituições na TLB e alternando operações para simular um uso mais realista da memória.

As configurações de utilizadas para os testes foram as seguintes: o tamanho da TLB unificada (*memorySize*) variou entre 2, 4 e 8 entradas; O número de ciclos por acesso de leitura foi definido como 1, e por escrita como 2; O tempo de ciclo foi configurado como 1 unidade. O algoritmo de substituição utilizado foi o FIFO ou o LRU, dependendo da simulação.

Os resultados (Figuras 3 e 4) mostram que aumentar o

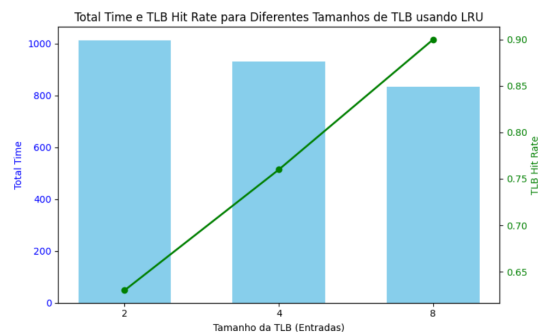


Fig. 3. Relação entre o aumento da TLB, taxa de hit e tempo total (LRU).

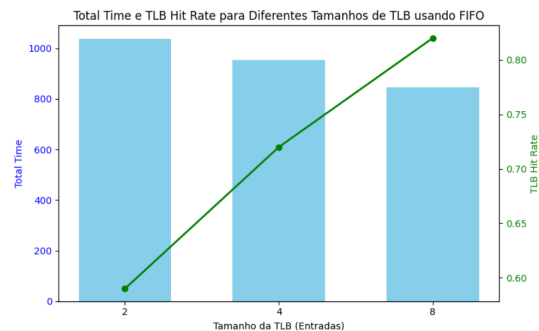


Fig. 4. Relação entre o aumento da TLB, taxa de hit e tempo total (FIFO).

tamanho da TLB melhora o desempenho — reduzindo o tempo total de execução e aumentando a taxa de acerto — independentemente da política adotada. No entanto, a política LRU apresentou desempenho superior à FIFO, alcançando taxas de acerto mais altas para os mesmos tamanhos de TLB. Isso ocorre porque a FIFO pode descartar entradas ainda úteis, enquanto a LRU “protege” os endereços que continuam sendo acessados repetidamente, como é o caso deste *trace* com forte localidade temporal.

Conclui-se que a presença de uma TLB, mesmo que pequena, é crucial, principalmente para a redução do tempo de execução. Neste cenário, o tempo registrado sem o uso de TLB foi de 1260.

3) Cenário 1.3: Número de Páginas: Analisou-se o impacto do número de páginas disponíveis na memória virtual (variando de 2 a 64) no tempo total de execução. O *trace* utilizado explorou a localidade espacial, acessando endereços próximos.

Observou-se que aumentar a quantidade de páginas na memória virtual reduz o tempo total de execução. Isso ocorre porque mais páginas permitem armazenar mais dados na memória principal simultaneamente, diminuindo a frequência de page faults. Menos page faults resultam em menos transferências de/para o disco e, conseqüentemente, menor tempo de execução. Embora aumentar o tamanho da página possa reduzir ainda mais o tempo, há o risco de aumentar a fragmentação interna se os dados carregados não forem to-

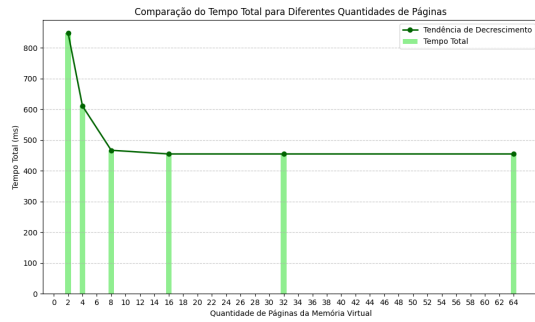


Fig. 5. Número de Page Faults por Tamanho de Página.

talmente utilizados. A conclusão principal é que um maior número de páginas na memória virtual é benéfico para reduzir o tempo de execução ao diminuir page faults.

B. Cache

A memória cache é uma memória menor e muito mais rápida que armazena cópias de dados que são acessados com frequência da memória principal. Ela explora dois tipos de localidade: a temporal e a espacial.

A eficiência da cache depende de vários fatores. Um deles é a política de substituição, que escolhe qual bloco será descartado quando a cache fica cheia. O tamanho total da cache também influencia: caches maiores armazenam mais dados, mas podem ter um tempo de acesso um pouco maior. Outro ponto importante é a associatividade, que determina quantos lugares diferentes um bloco de memória pode ocupar dentro da cache. Por fim, o tamanho da linha de cache afeta a quantidade de dados carregados de uma vez, o que pode ajudar a explorar ainda mais a localidade espacial.

1) Cenário 2.1: Políticas de Substituição e Tamanho da Cache: Neste cenário, foi avaliado o impacto das políticas de substituição First-In, First-Out (FIFO) e Least Recently Used (LRU), bem como da variação do tamanho da cache (de 4 a 64 linhas), sobre o número de cache misses e o tempo total de execução. A configuração experimental manteve o mesmo trace aleatório utilizado no estudo da memória virtual, garantindo a consistência da análise e permitindo observar os efeitos isolados das mudanças na arquitetura de cache.

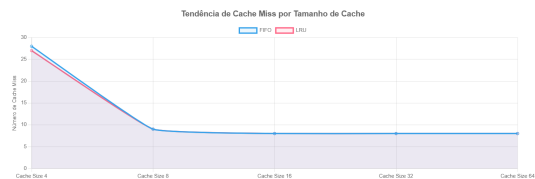


Fig. 6. Número de Cache Misses por Tamanho da Cache.

Devido à natureza aleatória do trace, o padrão de acessos não explorava fortemente a localidade temporal ou espacial, o que tornou o sistema sensível ao tamanho da cache disponível.

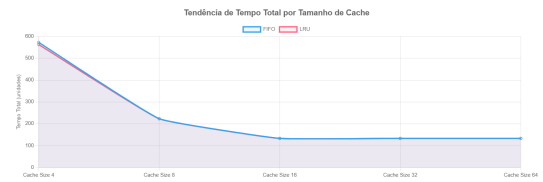


Fig. 7. Tempo de Cache Misses por Tamanho da Cache

Os resultados indicaram a presença de um ponto de saturação: o aumento do tamanho da cache de 4 para 8 linhas reduziu significativamente os cache misses, com uma queda aproximada de 70% no número de falhas. No entanto, ao dobrar novamente a capacidade de 8 para 16 linhas, a redução foi muito menor, em torno de 11%.

Além disso, expandir ainda mais a cache para 32 ou 64 linhas não trouxe ganhos adicionais. O número de misses e o tempo de execução permaneceram praticamente constantes, evidenciando um desperdício de recursos, já que a maior capacidade de armazenamento não se traduzia em melhora efetiva de desempenho para o padrão de acesso em questão. Esse comportamento reforça a importância de dimensionar a cache de acordo com a natureza da carga de trabalho, evitando superdimensionamento desnecessário.

Em relação às políticas de substituição, observou-se que a LRU apresentou apenas uma pequena vantagem sobre o FIFO, perceptível principalmente quando a cache era extremamente limitada (4 linhas). Como o trace era altamente aleatório, o benefício de manter os dados mais recentemente acessados foi minimizado, tornando o desempenho das duas políticas bastante semelhante na maioria dos cenários avaliados.

Esses resultados ilustram que, em cargas de trabalho sem forte localidade, o aumento descontrolado da cache e o uso de políticas mais complexas de substituição podem não trazer os benefícios esperados, exigindo uma análise criteriosa para otimizar o custo-benefício do sistema.

2) Cenário 2.2: Tamanho da Linha da Cache: Investigou-se o efeito do tamanho da linha da cache (variando de 1 a 32) no número de cache misses e no tempo de execução, utilizando uma memória principal de 256 unidades e cache de 64 unidades. O trace explorou a localidade espacial, onde o acesso a um dado tende a ser seguido por acessos a dados próximos.

Os resultados mostraram que um tamanho de linha muito pequeno prejudica o desempenho, gerando muitos cache misses. Conforme o tamanho da linha aumenta, o número de misses e o tempo diminuem devido à melhor exploração da localidade espacial. No entanto, a partir de um certo ponto, os ganhos tornam-se marginais ou inexistentes. No exemplo, passar de 8 para 16 trouxe uma melhora de apenas 3%, e de 16 para 32 não houve melhora aparente. Isso sugere um tamanho ótimo de linha que equilibra a busca antecipada de dados (prefetching) e a poluição da cache com dados não utilizados.

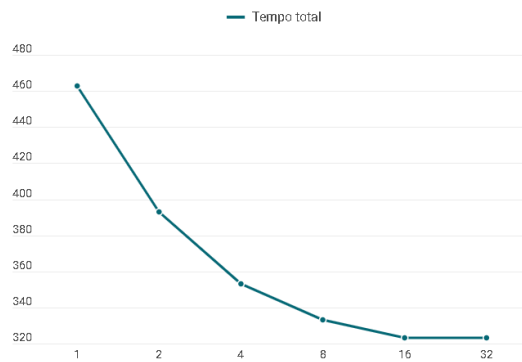


Fig. 8. Tempo de Execução por Tamanho de Linha.

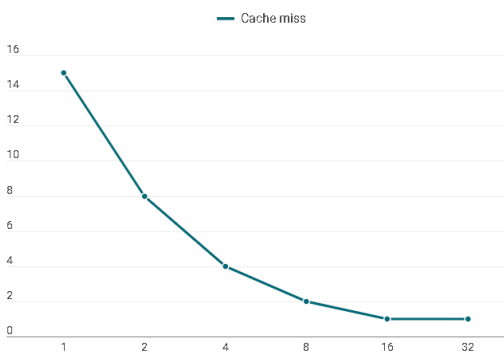


Fig. 9. Número de Cache Miss por Tamanho de Linha.

3) **Cenário 2.3: Associatividade:** Este cenário analisou o impacto da associatividade da cache no desempenho. Utilizou-se um trace com padrão de acesso repetitivo, explorando localidade temporal, para avaliar como diferentes níveis de associatividade lidam com conflitos de mapeamento.

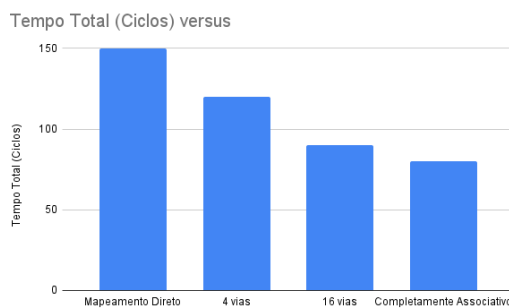


Fig. 10. Número de Page Faults por Tamanho de Página.

Os resultados demonstram uma correlação direta entre o nível de associatividade e a eficiência no tempo de execução. Conforme ilustrado na Figura 10, o mapeamento direto apresentou o maior tempo total de execução (em ciclos), evidenciando maior ocorrência de conflitos de mapeamento e, consequentemente, mais falhas de cache. À medida que a associatividade aumenta, passando por configurações de 4

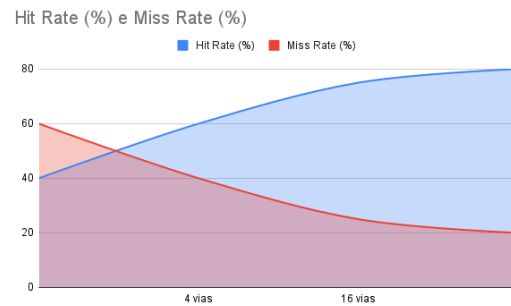


Fig. 11. Número de Page Faults por Tamanho de Página.

vias, 16 vias e chegando à cache completamente associativa, observa-se uma redução progressiva do tempo de execução, o que evidencia um melhor aproveitamento da cache.

Essa tendência é ainda mais clara quando analisamos a Figura 11, que apresenta as taxas de acerto (Hit Rate) e erro (Miss Rate) da cache em cada configuração. Nessa figura, nota-se que o aumento da associatividade resulta em uma elevação significativa da Hit Rate e uma queda acentuada da Miss Rate. Isso demonstra que caches com maior associatividade conseguem armazenar e recuperar dados com mais eficiência, reduzindo a frequência de acessos à memória principal. Essa melhoria é especialmente relevante em contextos com localidade temporal acentuada, como o utilizado neste cenário, onde os dados acessados tendem a ser reutilizados em curtos intervalos de tempo. Portanto, conclui-se que a elevação da associatividade não apenas reduz o tempo de execução, mas também melhora significativamente a taxa de acertos da cache, contribuindo para o desempenho geral do sistema.

V. CONCLUSÃO GERAL

A análise dos diferentes cenários demonstrou a complexa interdependência entre os parâmetros de configuração da memória virtual e da cache e o desempenho geral do sistema. Cada ajuste realizado mostrou impactos que nem sempre são lineares, evidenciando a necessidade de uma abordagem cuidadosa na escolha das configurações.

Para a memória virtual, a política LRU (Least Recently Used) mostrou-se geralmente superior à FIFO (First-In First-Out) nos testes realizados, pois consegue melhor capturar os padrões de reutilização de páginas em cargas de trabalho típicas. Um tamanho de página menor foi benéfico para o trace aleatório utilizado, uma vez que permite maior granularidade na alocação de memória e reduz o desperdício de espaço. Por outro lado, o aumento no número de páginas disponíveis foi eficaz para diminuir a quantidade de page faults, especialmente em cenários com maior pressão de memória. A utilização da TLB (Translation Lookaside Buffer) provou ser essencial para acelerar a tradução de endereços virtuais para físicos, com o desempenho do sistema melhorando de maneira quase linear com o aumento do tamanho da TLB.

No que diz respeito à memória cache, observou-se a existência de um ponto de saturação em relação ao seu tamanho:

a partir de certo valor, aumentos adicionais na capacidade da cache não resultaram em ganhos expressivos de desempenho. O tamanho da linha da cache se mostrou um fator crítico para a exploração da localidade espacial, uma vez que carregar blocos maiores de dados pode antecipar acessos futuros. No entanto, após determinado limite, o ganho marginal diminui e pode até introduzir ineficiências. A associatividade da cache demonstrou ser uma ferramenta eficaz para mitigar conflitos de mapeamento, reduzindo a taxa de substituições desnecessárias. Em contrapartida, níveis muito altos de associatividade podem introduzir maior complexidade de implementação e impactar o tempo de acesso.

Em suma, a otimização do sistema de memória envolve encontrar um equilíbrio delicado entre esses parâmetros, levando em consideração as características específicas das cargas de trabalho esperadas. Ajustes que favorecem uma aplicação podem ser prejudiciais para outra, e há sempre um trade-off entre desempenho, custo de hardware e complexidade de gerenciamento. Uma análise criteriosa dos padrões de acesso e uma avaliação experimental cuidadosa são fundamentais para alcançar um projeto eficiente e robusto.

Como trabalhos futuros, sugere-se a aplicação de técnicas adaptativas baseadas em aprendizado de máquina, capazes de ajustar dinamicamente os parâmetros da memória de acordo com o padrão de acesso observado. Além disso, seria relevante investigar o comportamento do sistema de memória sob cargas de trabalho mais variadas, incluindo aplicações de tempo real e sistemas paralelos, que impõem restrições e padrões de acesso distintos. Embora o simulador utilizado neste estudo não suporte tais cenários, a proposta permanece válida como uma diretriz para extensões futuras.

REFERENCES

- [1] A. Wierzbicki, N. Leibowitz, M. Ripeanu, and R. Wozniak, "Cache replacement policies revisited: the case of p2p traffic," in *IEEE International Symposium on Cluster Computing and the Grid, 2004. CCGrid 2004.*, 2004, pp. 182–189.
- [2] Y. Ding, H. Xu, P. Li, and J. Ding, "Review of virtual memory optimization in cloud environment," in *2016 International Conference on Intelligent Networking and Collaborative Systems (INCoS)*, 2016, pp. 53–58.
- [3] M. M. Elahresh and A. E. Ali Elaraby, "Achieving zero cache-miss based on the control flow graph," in *2021 IEEE 1st International Maghreb Meeting of the Conference on Sciences and Techniques of Automatic Control and Computer Engineering MI-STA*, 2021, pp. 375–380.